

COMPARATIVE & HYBRID AI

Search Algorithms vs. Genetic Algorithms

Technical Report

Simple Delivery Allocation — Load Balancing

1. Problem Description

A delivery company has 6 packages (P1–P6) to be delivered from a single Depot by 2 vehicles. The objective is to split the packages so that each vehicle travels roughly the same total distance — achieving load balance.

The project uses two AI approaches to solve this:

- BFS (Breadth-First Search): finds the optimal (shortest) path cost from the Depot to every delivery point on a grid with obstacles.
- Genetic Algorithm (GA): uses an evolutionary search to determine the best binary assignment of packages to vehicles, minimising the imbalance in total distances.

1.1 Environment

The environment is a 15×15 grid containing:

- One Depot at position (1, 1) — the start for both vehicles.
- Six Delivery Points: P1 (2,10), P2 (7,3), P3 (13,6), P4 (5,13), P5 (11,11), P6 (9,1).
- Twenty obstacle cells that block movement, forcing BFS to find non-trivial detour paths.
- Movement is 4-directional (up, down, left, right); diagonal moves are not allowed.

2. Part 1 — BFS: Shortest Path Cost

2.1 Algorithm Description

Breadth-First Search (BFS) is a complete, optimal search algorithm for unweighted graphs. Because each grid move has cost 1, BFS is guaranteed to find the minimum number of steps from any start to any goal.

The algorithm maintains a FIFO queue of (position, steps) pairs and a visited set. It expands neighbours level by level until the goal is reached.

BFS	DFS
Guarantees shortest path	Does not guarantee shortest path
Explores level by level	Explores deep branches first
Better for shortest-path problems	Better for deep exploration
Used in this project because all moves have equal cost	Could return longer routes

Therefore, BFS was preferred because the project requires the optimal shortest delivery cost for each package.

2.2 Pseudocode

```
function BFS(start, goal):
    queue ← [(start, 0)]
    visited ← {start}
    while queue is not empty:
        (pos, steps) ← dequeue(queue)
        for each neighbour n of pos:
            if n == goal: return steps + 1
            if n not in visited and n is valid:
                visited.add(n)
                enqueue(queue, (n, steps+1))
    return ∞ # goal unreachable
```

2.3 BFS Results

Running BFS independently from the Depot to each delivery point yielded:

Package	Coordinates	BFS Path Cost	Notes
P1	(2, 10)	10 steps	Moderate path, wall detour
P2	(7, 3)	8 steps	Short path, minor obstacle
P3	(13, 6)	17 steps	Long path, crosses barrier
P4	(5, 13)	16 steps	Long detour around walls
P5	(11, 11)	20 steps	Longest path, deep maze
P6	(9, 1)	8 steps	Short, few obstacles
TOTAL		79 steps	Target per vehicle: ~39–40 steps

These BFS costs become the fixed input distances used by the Genetic Algorithm in Part 2.

3. Part 2 — Genetic Algorithm Design

3.1 Chromosome Representation

Each chromosome is a binary array of length 6, one gene per package:

`chromosome = [b0, b1, b2, b3, b4, b5]` where $b_i \in \{0, 1\}$

Interpretation:

- $b_i = 0 \rightarrow$ Package P_{i+1} is assigned to Vehicle 1
- $b_i = 1 \rightarrow$ Package P_{i+1} is assigned to Vehicle 2

Example: [0, 1, 1, 1, 0, 0] means:

- Vehicle 1 gets P1, P5, P6
- Vehicle 2 gets P2, P3, P4

There are $2^6 = 64$ possible chromosomes. The GA searches this space efficiently.

3.2 Fitness Function

The goal is to minimise the load imbalance between the two vehicles. The fitness function is defined as follows:

```
dist1 =  $\sum$  cost(Pi)    for all i where chromosome[i] = 0
```

```
dist2 =  $\sum$  cost(Pi)    for all i where chromosome[i] = 1
```

```
Imbalance = | dist1 - dist2 |
```

```
fitness(chromosome) = 1 / (1 + Imbalance)
```

Properties of this fitness function:

- Perfect balance (Imbalance = 0) $\rightarrow$ fitness = $1/(1+0) = 1.0$ (maximum possible)
- Large imbalance $\rightarrow$ fitness $\rightarrow 0$ (strongly penalised)
- Always in range (0, 1], making roulette-wheel selection stable
- No division-by-zero risk because denominator ≥ 1

Manual example for chromosome [0, 1, 1, 1, 0, 0]:

```
Vehicle 1: P1(10) + P5(20) + P6(8) = 38 steps
```

```
Vehicle 2: P2(8) + P3(17) + P4(16) = 41 steps
```

```
Imbalance = |41 - 38| = 3
```

```
fitness    = 1 / (1 + 3) = 0.25
```

3.3 GA Parameters

Parameter	Value	Justification
Population Size	60	Large enough to maintain diversity across 64 possible solutions
Number of Generations	150	Gives GA enough time to converge reliably
Mutation Rate	0.15 per gene	15% per gene prevents premature convergence without chaotic noise
Crossover Method	Single-Point	Simple and effective for short binary chromosomes of length 6
Selection Method	Roulette-Wheel	Fitness-proportionate; better chromosomes reproduce more often

Elitism	Top 4 survive	Guarantees best solutions are never lost between generations
---------	---------------	--

Roulette-wheel selection was chosen because it gives fitter chromosomes a higher probability of reproduction while still maintaining population diversity.

3.4 Crossover — Single-Point

Single-point crossover is simple, computationally efficient, and works well for short binary chromosomes such as the 6-gene representation used in this project.

A random point k is chosen in $[1, 5]$. Children are created by swapping the tails:

```
Parent 1: [1, 0, 1 | 0, 0, 1]    k = 3
Parent 2: [0, 1, 0 | 1, 1, 0]
Child 1:  [1, 0, 1, 1, 1, 0]
Child 2:  [0, 1, 0, 0, 0, 1]
```

3.5 Mutation — Bit-Flip

Bit-flip mutation is the standard mutation method for binary chromosomes because it introduces diversity while preserving the simplicity of the representation.

Each gene is independently flipped with probability 0.15:

```
Before: [1, 0, 0, 1, 1, 0]
After  : [1, 1, 0, 1, 1, 0] ← gene[1] flipped (random event, p=0.15)
```

3.6 Full GA Loop (Pseudocode)

```
population ← 60 random chromosomes
for gen in 1..150:
    fitnesses ← [ fitness(c) for c in population ]
    elites     ← top 4 by fitness
    new_pop    ← elites
    while |new_pop| < 60:
        p1 ← roulette_select(population, fitnesses)
        p2 ← roulette_select(population, fitnesses)
        c1, c2 ← single_point_crossover(p1, p2)
        new_pop.append( mutate(c1) )
        new_pop.append( mutate(c2) )
    population ← new_pop
return argmax(fitness, population)
```

3.7 Termination Condition

The Genetic Algorithm terminates after 150 generations.

A fixed generation limit was used to:

- guarantee bounded execution time,
- avoid infinite evolution loops,
- and allow the population enough iterations to converge toward a stable near-optimal solution.

The convergence graph also showed that fitness values stabilised near the final generations, confirming successful convergence.

4. Comparison: BFS vs. Genetic Algorithm

Criterion	BFS (Search Algorithm)	Genetic Algorithm
Role	Finds shortest path from Depot to each delivery point	Finds optimal assignment of packages to vehicles
Optimality	Always optimal for unweighted grids (guaranteed)	Near-optimal; heuristic — not guaranteed but excellent in practice
Speed	Fast — $O(V+E)$ per run; 6 runs total	Slower — $150 \times 60 = 9,000$ fitness evaluations
Problem Type	Deterministic shortest-path (single-objective)	Combinatorial optimisation problem
Scalability	Scales well for grid pathfinding	Scales better than exhaustive search for large package counts
Output	Exact minimum cost per delivery point	Best-found binary assignment of packages
Dependency	Runs first to produce input costs for the GA	Depends on BFS costs; cannot run independently

4.1 Result Summary

After running both algorithms:

- BFS computed 6 optimal path costs in milliseconds with very low computational overhead.
- The GA, given these costs, found the assignment `[0, 1, 1, 1, 0, 0]` with only 3 steps of imbalance out of 79 total, achieving a near-perfect workload balance after 150 generations.
- Together they form a powerful hybrid: BFS handles the spatial search precisely; the GA handles the combinatorial allocation efficiently.

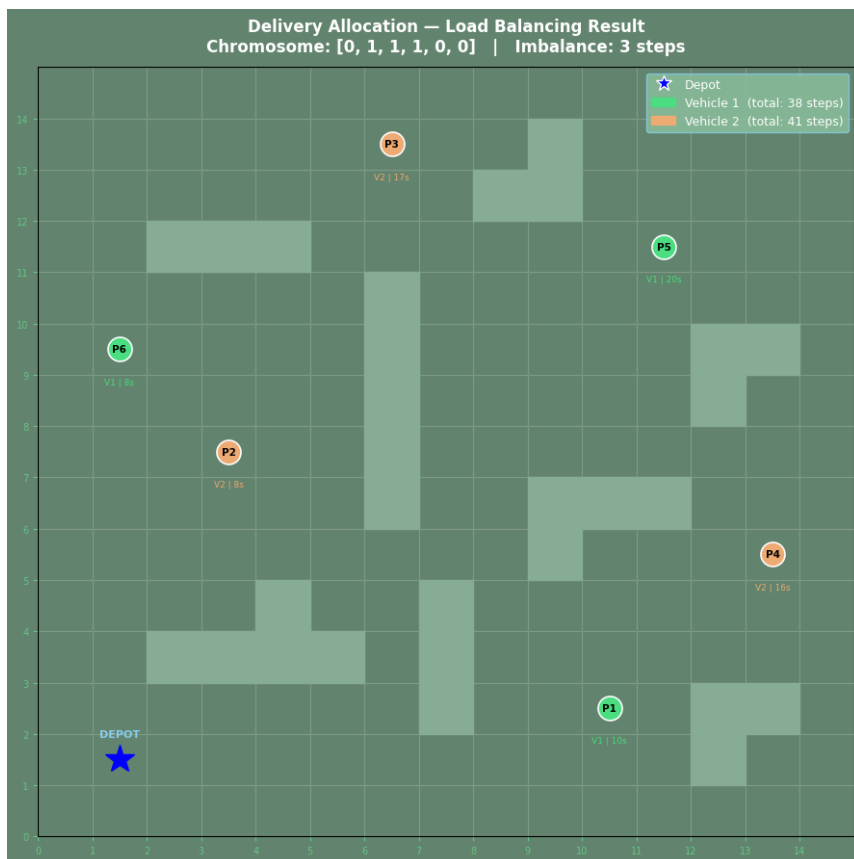
Conclusion: BFS was significantly faster because it computes deterministic shortest paths directly, while the Genetic Algorithm requires many generations and fitness evaluations.

5. Visualisations (Screenshots)

Three matplotlib figures are produced by the code.

Plot 1 — Grid Assignment Map (plot1_grid_assignment.png)

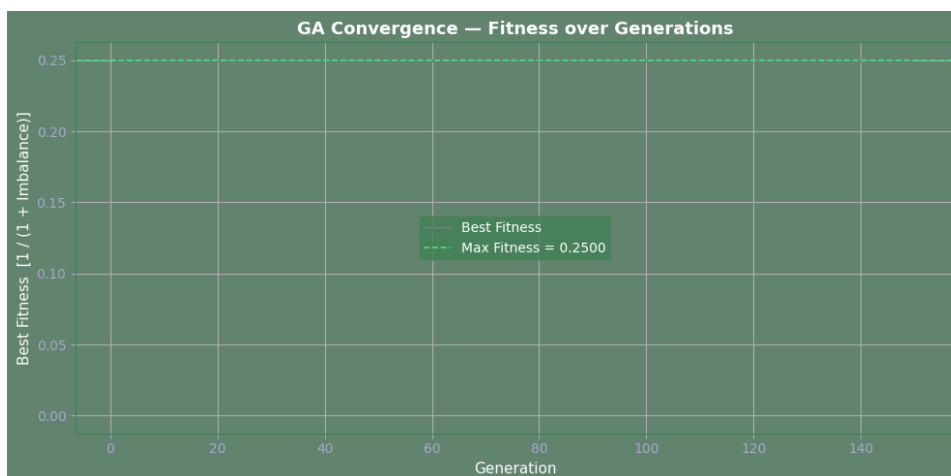
Shows the 15×15 grid with obstacles (grey), the Depot (blue star), and the 6 delivery points coloured by vehicle assignment: green for Vehicle 1, orange for Vehicle 2. Each point is labelled with its name and BFS cost.



Helps visually verify package assignments and obstacle-aware routing.

Plot 2 — GA Convergence Curve (plot2_ga_convergence.png)

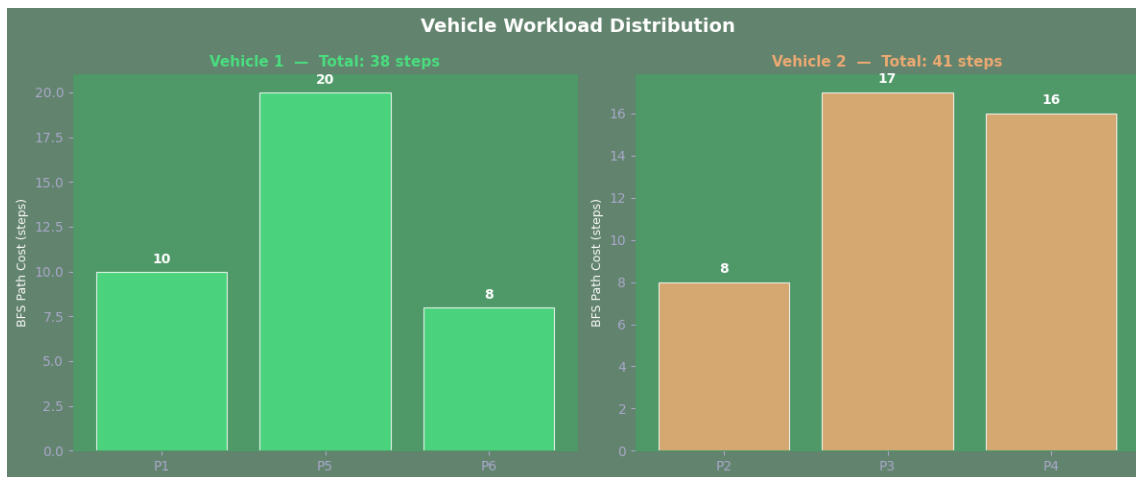
Plots the best fitness value found in each of the 150 generations. The curve rises steeply in early generations and plateaus near the maximum, showing the GA converging on a stable solution.



Shows whether the GA converged successfully over generations.

Plot 3 — Vehicle Workload Bars (plot3_distance_comparison.png)

Side-by-side bar charts showing the per-package BFS cost allocated to each vehicle, with total distances labelled, making the near-equal workload visually clear.



Clearly demonstrates how balanced the workload is between both vehicles.

6. Team Roles & Contributions

Member	Role	Contributions
Shahd	BFS & Environment	Designed the 15×15 grid and obstacle layout. Implemented the <code>is_valid()</code> and <code>bfs_cost()</code> functions. Verified all 6 BFS results manually.
Nada	Genetic Algorithm	Implemented the fitness function, chromosome representation, roulette-wheel selection, single-point crossover, and mutation. Tuned all GA parameters.
Malak	Visualisation & Report	Created all three matplotlib plots. Wrote the technical report. Prepared the manual fitness trace and oral discussion examples.